

1. Project Title

UPI Payment Gateway Simulator

A backend-driven payment processing system that simulates real-world UPI payment gateway functionality such as merchant onboarding, payment processing, callbacks, refunds, and settlements.

2. Problem Statement

Digital payment systems such as Razorpay, PhonePe, and Paytm handle millions of UPI transactions daily. These systems require high reliability, clear transaction state management, extensible payment modes, and real-time communication between merchants and payment gateways.

However, understanding their backend working is difficult for students due to system complexity.

This project aims to design and implement a **UPI Payment Gateway Simulator** that demonstrates how payment gateways operate internally using proper **Object Oriented Design, UML models, Design Patterns, and Architecture principles**.

3. Objectives

1. Simulate UPI payment lifecycle.
 2. Support merchant onboarding and authentication.
 3. Process payments using multiple strategies.
 4. Maintain transaction states.
 5. Handle callbacks using the Observer pattern.
 6. Support refund and settlement operations.
 7. Demonstrate clean OOAD design using UML and patterns.
-

4. Scope of the System

The system will:

- Act as a **Payment Gateway Backend**
 - Expose REST APIs for merchants.
 - Provide a React dashboard for merchants.
 - Store transaction data in the database.
 - Simulate UPI success/failure scenarios.
-

5. Actors

Actor	Description
Merchant	Initiates payment requests
Customer	Pays using UPI
Payment Gateway	Core system
Bank Simulator	Simulates bank responses
Admin	Manages merchants

6. Functional Requirements

Major Use Cases

1. Merchant Onboarding
2. Payment Initiation
3. Payment Verification
4. Callback Notification
5. Refund Processing
6. Transaction Settlement

Minor Use Cases

1. View Transactions
 2. Transaction Status Tracking
 3. Settlement Reports
 4. Merchant Profile Management
-

7. Non-Functional Requirements

- Scalable design
 - Secure APIs
 - Modular architecture
 - Maintainable code
 - UML compliance
-

8. Architecture

MVC Architecture (As per guidelines)

React Frontend → Controller → Service → Repository → Database

Layers

Layer	Responsibility
Controller	API endpoints
Service	Business logic
Repository	DB access
Model	Entities
DTO	API communication

9. Database Design

Entities:

- Merchant
 - Transaction
 - Refund
 - Settlement
 - CallbackLog
-

10. UML Diagrams

You must create:

1. Use Case Diagram

Actors → Merchant, Admin

Use cases → Initiate Payment, Refund, View Settlement

2. Class Diagram

Key classes:

PaymentRequest

PaymentResponse

Transaction

Merchant

Refund

Settlement

PaymentProcessor (interface)

UPIProcessor

CardProcessor

NetBankingProcessor

CallbackObserver

MerchantCallback

AdminCallback

3. Activity Diagram – Payment Flow

Start → Merchant sends request

→ Validate merchant

→ Create transaction

→ Process payment

→ Update status

→ Send callback

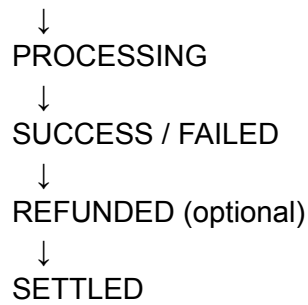
→ End

4. State Diagram – Transaction

CREATED

↓

INITIATED



This diagram is **very important for marks**.

11. Design Patterns Used

As required: Creational + Structural + Behavioral.

1. Factory Pattern (Creational)

Used to create payment processors.

```
PaymentProcessor processor = PaymentFactory.getProcessor("UPI");
```

2. Strategy Pattern (Behavioral)

Different payment methods:

UPIProcessor
CardProcessor
NetBankingProcessor

All implement:

PaymentProcessor

3. Observer Pattern (Behavioral)

Callbacks to merchants.

Transaction → notifies → MerchantCallback

4. Singleton Pattern (Creational)

Used for Configuration Manager / DB Config.

12. Design Principles Used

Principle	Usage
SRP	Separate services
OCF	New payment methods without modifying code
LSP	Processor substitution
ISP	Small interfaces
DIP	Interface based coding

13. Transaction Lifecycle Explanation

1. Merchant calls `/initiate-payment`
 2. System creates transaction with status CREATED
 3. Factory selects processor
 4. Strategy processes payment
 5. Bank simulator responds
 6. Status updated
 7. Observer triggers callback
 8. Merchant receives response
 9. Settlement scheduled
-

14. API Endpoints

API	Method
/merchant/register	POST
/payment/initiate	POST
/payment/verify	GET
/payment/refund	POST
/payment/status	GET
/settlement/report	GET

15. React Frontend

Screens

1. Merchant Registration
2. Payment Initiation
3. Transaction History
4. Refund Request
5. Settlement Dashboard

React uses:

- Axios
 - React Router
 - Tailwind / CSS
-

16. Implementation Steps

Phase 1 – Design

1. Finalize use cases
 2. Draw UML diagrams
 3. Assign use case per student
 4. Identify patterns
 5. Architecture design
-

Phase 2 – Backend

1. Create Spring Boot project
 2. Create entities
 3. Implement repositories
 4. Implement service logic
 5. Implement patterns
 6. Add APIs
 7. Add exception handling
-

Phase 3 – Frontend

1. Create React app
 2. Create pages
 3. Connect APIs
 4. Display results
-

Phase 4 – Integration

1. Connect frontend with backend
 2. Test flows
 3. Fix bugs
-

17. Team Member Distribution (Example)

Member	Use Case
Member 1	Merchant Onboarding
Member 2	Payment Processing
Member 3	Refund Handling
Member 4	Settlement + Reports

Each owns **UI + backend + UML** of their use case.

18. Github Repository Structure

backend/
frontend/
docs/
uml/

19. Report Structure (As per guideline)

- 1. Title Page
 - 2. Problem Statement
 - 3. Requirements
 - 4. UML Models
 - 5. Architecture
 - 6. Design Patterns
 - 7. Design Principles
 - 8. Screenshots
 - 9. Github Link
 - 10. Individual Contributions
-

20. Why This Project Scores High

Criteria	Covered
UML	Rich diagrams
MVC	Spring Boot
Patterns	4 patterns
Principles	4 principles
Complexity	Real payment domain
Industry relevance	Very high

21. Next Step for You

If you want, I can now give:

- Exact UML diagrams content for drawing
- Class diagram with attributes & methods
- State diagram transitions
- Sequence diagram
- Spring Boot package structure
- React component structure
- Sample API JSON formats
- PPT content for presentation